

*** REGRESSION** to relate input variables to the output variable, to either predict or understand

↳ Regression Dataset: (x_n, y_n) , y_n is n^{th} output
 x_n = vector of D inputs
 ↳ Data-size: # of pairs N ↳ D = Dimensionality.
 ↳ Regression fct: $y_n \approx f(x_n)$ $\forall n$
 ⚠ Correlation $\neq$ causation

↳ Linear Regression: model that assumes a linear relationship between inputs and the output

↳ fct: $y_n \approx f(x_n) = w_0 + w_1 x_n$
 $\rightarrow \tilde{w} = (w_0, w_1)$ are parameters of the model.

↳ multivariate lin. reg: $f(x_n) = w_0 + x_n^T \tilde{w} = \tilde{x}_n^T \tilde{w}$

• Fitting the model: finding $\tilde{w} = [w_0, w_1, \dots, w_D]$
 ↳ need optimization algo

• The $D > N$ pb: more variables than data $\rightarrow$ overparametrized
 ↳ solution: regularization

*** COST FUNCTIONS** quantifies how well our model performs

↳ desirable properties: • symmetry around 0 (if y real)
 • penalize "large" & "very large" mistakes similarly

• MSE $(w) = \frac{1}{N} \sum_{n=1}^N [y_n - f_w(x_n)]^2$ Δ MSE not good w/ outliers

• MAE $(w) = \frac{1}{N} \sum_{n=1}^N |y_n - f_w(x_n)|$ $\leftarrow$ convex

• Loss: $e = y - xw$

• RMSE $= \sqrt{2 \cdot \text{MSE}(w)}$

• Hinge $(z, y) = [1 - yz]^+$

*** CONVEXITY** a line joining 2 points never intersects w/ the fct elsewhere

↳ fct is convex $\Leftrightarrow$ $h(u)$, $u \in X$ convex if, for any $u, v \in X$ & $0 \leq \lambda \leq 1$:
 $h(\lambda u + (1-\lambda)v) \leq \lambda h(u) + (1-\lambda)h(v)$ $\leftarrow$ strictly convex or $h''(x) \geq 0$

$\Rightarrow$ a strictly convex fct has a unique global minimum w^*

$\Rightarrow$ for convex fcts, every local minimum is a global minimum

• Joint convex $\Leftrightarrow$ positive semi-definite Hessian $= \begin{bmatrix} \frac{\partial^2 L}{\partial w_0^2} & \frac{\partial^2 L}{\partial w_0 \partial w_1} \\ \frac{\partial^2 L}{\partial w_1 \partial w_0} & \frac{\partial^2 L}{\partial w_1^2} \end{bmatrix}$

*** OPTIMIZATION**

• Given cost fct $L(w)$, we want w^* which minimises cost:
 $\min_w L(w)$ subject to $w \in \mathbb{R}^D$

• Grid Search: brute force, compute cost over all w , take best

*** SMOOTH OPTIM** Gradient: gradient at a point is the slope of the tangent to the fct (at pt). It points to the dir. of largest increase of the function.

$\nabla L(w) = \left[\frac{\partial L(w)}{\partial w_0}, \dots, \frac{\partial L(w)}{\partial w_D} \right]^T \in \mathbb{R}^D$
 $\gamma > 0$: step-size

• Gradient Descent: $w^{(t+1)} = w^{(t)} - \gamma \nabla L(w^{(t)})$
 ↳ to minimise fct, we iteratively take a step in the opposite direction of the gradient.

• GD for linear MSE: $\nabla L(w) = -\frac{1}{N} X^T e$ $e = y - Xw$

• Stochastic Gradient Descent: $\nabla L(w) = \frac{1}{N} \sum_{n=1}^N \nabla L_n(w)$
 $w^{(t+1)} = w^{(t)} - \gamma \nabla L_n(w^{(t)}) \rightarrow L(w) = \frac{1}{N} \sum_{n=1}^N L_n(w)$

$\rightarrow$ Idea: cheap but unbiased estimate of the gradient!

In expectation over the random choice of n , we have:
 $E[\nabla L_n(w)] = \nabla L(w) \rightarrow$ true gradient direction

• Minibatch SGD: $g = \frac{1}{B} \sum_{n \in B} \nabla L_n(w^{(t)})$
 $w^{(t+1)} = w^{(t)} - \gamma g$, $B \subseteq [N]$

*** NON-SMOOTH OPTIM** convexity for differentiable fcts:
 i.e. the fct must always lie above its linearization $\{L(u) \geq L(w) + \nabla L(w)^T (u-w) \forall u, w\}$

*** SUBGRADIENTS**

• $\tilde{g} \in \mathbb{R}^D$ st: $L(u) \geq L(w) + \tilde{g}^T (u-w) \forall u$
 $=$ subgradient to the fct L at w .

• If L is convex & diff. at $w \rightarrow$ only subgradient at w is $g = \nabla L(w)$
 ↳ MAE: $g(w) = \frac{1}{N} \sum_{n=1}^N \text{Sign}(y_n - f_w(x_n)) (y_n - f_w(x_n))$

$\Rightarrow$ subgradient descent: $w^{(t+1)} = w^{(t)} - \gamma \tilde{g}$
 $\Rightarrow$ stochastic subgradient descent {some, $\tilde{g}$ being a subgradient to the randomly selected x_n at $w^{(t)}$ } $\in \mathbb{R}^D$ constraint set

*** CONSTRAINED OPTIMIZATION**

• Optim pb of form: $\min_w L(w)$, subject to $w \in C$
 ↳ solve by: A) Projected Gradient Descent
 B) Transform it into an unconstrained pb

• Convex Sets: C convex $\Leftrightarrow$ the line segment between any two points of C lies in C , i.e., if for any $u, v \in C$ and any θ with $0 \leq \theta \leq 1$,
 $\rightarrow \theta u + (1-\theta)v \in C$

• Properties: • Intersections of convex sets are convex
 • Projections onto convex sets are unique
 $\rightarrow P_C(w) = \arg \min_{v \in C} \|w - v\|$

• Projected Gradient Descent
 • Idea: add a projection onto C after every step.
 $\rightarrow P_C(w) = \arg \min_{v \in C} \|w - v\|$

$\rightarrow$ Update rule: $w^{(t+1)} = P_C [w^{(t)} - \gamma \nabla L(w^{(t)})]$

$\rightarrow$ Projected SGD: SGD step + projection step

• Constrained $\rightarrow$ Unconstrained
 ↳ Use penalty functions instead of directly solving $\min_{w \in C} L(w)$

• brick wall: $l_C(w) = \begin{cases} 0 & w \in C \\ \infty & w \notin C \end{cases} \Rightarrow \min_{w \in \mathbb{R}^D} L(w) + l_C(w)$
 (indicator fct)

• Penalize error: $C = \{w \in \mathbb{R}^D | Aw = b\} \Rightarrow \min_{w \in \mathbb{R}^D} L(w) + \lambda \|Aw - b\|^2$

• Linearized Penalty functions

• Implementation issues (for gradient methods)
 • Stopping criteria
 • optimality
 • Step-size selection

*** Least squares:** $w^* = (X^T X)^{-1} X^T y$
 $\frac{\partial L(w)}{\partial w} = 2X^T (y - Xw)$; $\frac{\partial^2 L(w)}{\partial w^2} = 2X^T X$
 $\rightarrow$ to obtain normal eqs, X^T show the pb is convex
 $\rightarrow$ then use the fact that $\nabla^2 L(w^*) = 0$

• Normal equations for linear regression: $X^T (y - Xw) = 0$

• Rank Deficiency

• Max likelihood = a probabilistic model for LS
 $\rightarrow y_n = x_n^T w + \epsilon_n$ $\rightarrow$ zero-mean gaussian noise $\rightarrow w$ unknown

↳ log likelihood: $L_L(w) = -\frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - x_n^T w)^2 + \text{const}$
 $\arg \min_w L_{\text{MSE}}(w) = \arg \max_w L_L(w) \rightarrow \text{MLE}$

• MLE properties: consistent, asymptotically normal, efficient

*** UNDERFITTING**: model is too limited, can't find a good fit
 $\rightarrow$ solution: augmented feature vectors
 $y_n \approx w_0 + w_1 x_n + w_2 x_n^2 + \dots + w_D x_n^D = \phi(x_n)^T w$

• OVERFITTING: $\rightarrow$ model is too rich, fits to noise in training data

↳ SOLUTION: REGULARIZATION
 ↳ penalize complex models & favor simpler ones

$\min_w L(w) + \Omega(w) \rightarrow$ regularizer $\|w\|_2^2 = \sum w_i^2$

• Ridge regression: $\Omega(w) = \lambda \|w\|_2^2$
 $\min_w \frac{1}{2N} \sum_{n=1}^N [y_n - x_n^T w]^2 + \lambda \|w\|_2^2$
 $w_{\text{ridge}}^* = (X^T X + \lambda I)^{-1} X^T y$ $\lambda = 2\lambda N$

• L1-regularization: $\Omega(w) = \lambda \|w\|_1 = \lambda \sum |w_i|$

• Ellipse equation: $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$

• Log likelihood: $L = \sum \log(\text{poly}(\theta))$

• DFT: $\hat{z}[k] = \sum_{n=0}^{N-1} z[n] \exp(-\frac{2\pi i}{N} kn)$

• IDFT: $z[n] = \frac{1}{N} \sum_{k=0}^{N-1} \hat{z}[k] \exp(\frac{2\pi i}{N} kn)$

• Estimator unbiased $\Leftrightarrow E[\hat{\theta}] = \theta$

• L_0 norm: highest value, L_1 norm: # of non zero values
 • L_1 norm: MAE, L_2 : MSE

*** Model Selection** error decreases as $O(\frac{1}{\sqrt{N}})$

• True error: $L(f) = E_{y, x \sim D} [L(y, f(x))]$

• Test error: $L_{\text{test}}(f) = \frac{1}{S_{\text{test}}} \sum_{(y_n, x_n) \in S_{\text{test}}} L(y_n, f(x_n))$

• Generalization error: $|L(f) - L_{\text{train}}(f)|$

*** CROSS-VALIDATION**

*** BIAS-VARIANCE DECOMPOSITION**

• Bias: diff between avg prediction & correct value
 ↳ high bias $\rightarrow$ oversimplified model $\rightarrow$ high error

• Variance: spread of our data
 ↳ high var $\rightarrow$ overfits train data $\rightarrow$ high test error

*** CLASSIFICATION** $\rightarrow$ relates $\tilde{x}$ to y , but y can only take on discrete values (line regression)

• Classifier: divides input space into class regions
 ↳ can be linear or non linear
 ↳ decision boundaries

• Classification not just special case of regression

↳ Techniques: smoothing kernels, SVMs, logistic regression, neural networks

Bayes classifier: $f^*(x) = 1$ if $\eta(x) > 1/2$ $\rightarrow$ MAP criterion
 ↳ minimise prob of misclassification: $g(x) = \arg \min_y p(y|x)$

↳ in practice, $p(x, y)$ unknown so use data to learn distrib $p(g(x)) = y = \int p(y|x) p(g(x)|x) dx$

1) log. reg.
 2) quantize to a binary value $\rightarrow$ in odds to fit $(-\infty, \infty)$ values to $\{0, 1\}$

• Logistic Regression
 log fct: $\sigma(z) = \frac{e^z}{1 + e^z}$
 $\rightarrow p(1|x, w) = \sigma(x^T w + w_0)$ w_0 allows us to shift transition point along w

minimise loss: $L(w) = \sum_{n=1}^N \ln [1 + \exp(-x_n^T w)] - y_n x_n^T w$
 $w^* = \arg \min_w L(w)$ Opti: $\nabla L(w) = X^T [\sigma(Xw) - y]$

• Lemma: $L(w) = \sum_{n=1}^N \ln [1 + \exp(x_n^T w)] - y_n x_n^T w$ is convex in w
 $\rightarrow$ apply gradient descent: $w^{(t+1)} = w^{(t)} - \gamma \nabla L(w^{(t)})$

• Newton method = second order method $H(w) = X^T S X$
 $\rightarrow w^{(t+1)} = w^{(t)} - \gamma^{(t)} H^{(t)-1} \nabla L(w^{(t)})$ $S = \sigma(x_n^T w) [1 - \sigma(x_n^T w)]$
 $\rightarrow \nabla L(w^*) + H(w^*) (w - w^*) = 0$

• Regularized logistic regression be log reg has issues with linearly separable data
 $\arg \min_w - \sum_{n=1}^N \ln p(y_n | x_n^T w) + \frac{\lambda}{2} \|w\|^2$ $\rightarrow$ added penalty term
 ↳ predictor converges to max-margin solution

• Exponential family $p(y|\eta) = h(y) \exp[\eta^T \phi(y) - A(\eta)]$
 ↳ natural param η $\rightarrow$ sufficient stat $\phi(y)$

We require: $\int p(y|\eta) dy = 1 \rightarrow \int h(y) \exp[\eta^T \phi(y)] dy = e^{A(\eta)}$

• Bernoulli: $p(y|\eta) = \exp(\eta y) / (1 + \exp(\eta))$

• Gaussian: $p(y|\eta) = \exp(-\frac{\eta^2}{2}) / \sqrt{2\pi}$ $\rightarrow$ Poisson: $p(y|\mu) = \frac{\mu^y e^{-\mu}}{y!}$

• Properties: $\rightarrow A(\eta)$ is convex
 $\rightarrow \text{Var}(\eta) = E[\phi(y)]$; $\text{Var}(A(\eta)) = \text{Var}[\phi(y)]$

• Link function: η & μ linked: $\eta = g(\mu) \Leftrightarrow \mu = g^{-1}(\eta)$

• Generalized Linear Models: $p(y|x, w) = h(y) e^{\eta^T w(y) - A(\eta)}$

• k-Nearest Neighbour $f_{\text{NN}, k}(x) = \frac{1}{k} \sum_{n=1}^k y_{n^{(k)}}(x)$
 $\rightarrow$ large bias, small var $\rightarrow$ small bias, large variance

• large k: simple model, small k : complex model
 $\rightarrow$ need exp more data per extra dimension $d, N \rightarrow$ for d -dim data $\rightarrow$ need N^d data

• Curse of dimensionality $\rightarrow$ distance becomes "irrelevant"
 (dist: $\frac{d-1}{d} \rightarrow 1$ as $d \rightarrow \infty$)

*** SVMs**

Optimization pb: $\min_w \sum_{n=1}^N [1 - y_n x_n^T w] + \frac{\lambda}{2} \|w\|^2$ (7.1)
 using hinge loss

• The margin: region where our prediction is not yet "sufficiently" confident $\rightarrow$ width $= \frac{2}{\|w\|}$

• Optimal w : 1) A separating hyperplane in the margin
 2) A scaling of w s.t. no data point is in the margin
 3) Maximised margin

hard-margin formulation: linearly separable data
 ↳ requires separating hyperplane & maximal margin
 ↳ we can use SGD w/ surrogate to optimize: 0/1 loss minimization
 ↳ no obvious probabilistic interpretation of SVM
 • SVM issues: extension of multiclass is non trivial

• Duality • Goal: minimize $L(w) = \max G(w, \alpha)$
 equality if G is concave in w , convex in α and w, α domains are convex & convex
 • dual-diff. quadratic problem
 • normal then
 • α in space

* Neural Networks

learn best features from data
computation goes from left to right

Feedforward network: $\rightarrow$ no feedback loop

• $x_j^{(L)} = \sigma(\sum_i w_{ij}^{(L)} x_i^{(L-1)} + b_j^{(L)})$
activation function bias term

• **Barron theorem:** a sufficiently "smooth" fct can be approximated by a neural network w/ one hidden layer & the approximation error goes down like $1/\sqrt{n}$ over the number of nodes in the hidden layer. (fast convergence)

• Use SGD efficiently with **back propagation** by computing all the derivatives jointly.

• We have: $x^{(L)} = f^{(L)}(x^{(L-1)}) = \sigma((w^{(L)})^T x^{(L-1)} + b^{(L)})$

$\Rightarrow y = f(x) = f^{(L)} \circ \dots \circ f^{(1)}(x^{(0)}) \Rightarrow \mathcal{L} = \frac{1}{N} \sum_{n=1}^N (y_n - f^{(L)}(x_n^{(0)}))^2$

• **The backpropagation algorithm**

$\rightarrow$ Goal: compute $\frac{\partial \mathcal{L}}{\partial w_{ij}^{(L)}}$, $\frac{\partial \mathcal{L}}{\partial b_j^{(L)}}$

• Forward pass: $x^{(L)} = \sigma(z^{(L)})$
 $z^{(L)} = (w^{(L)})^T x^{(L-1)} + b^{(L)}$

• Backward pass: set $\delta^{(L+1)} = -2(y_n - x^{(L+1)}) \sigma'(z^{(L+1)})$

$g^{(L)} = \frac{\partial \mathcal{L}}{\partial z^{(L)}} = (w^{(L+1)})^T \delta^{(L+1)} \odot \sigma'(z^{(L)})$

• Finally: $\frac{\partial \mathcal{L}}{\partial w_{ij}^{(L)}} = \delta_j^{(L)} x_i^{(L-1)}$; $\frac{\partial \mathcal{L}}{\partial b_j^{(L)}} = \delta_j^{(L)}$

$\rightarrow$ Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$ $\rightarrow$ ReLU: $\max\{0, x\}$

$\rightarrow$ Tanh: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$

$\rightarrow$ Leaky ReLU: $f(x) = \max\{x, \alpha x\}$ $\rightarrow$ Maxout: $f(x) = \max\{x^1 w_1 + b_1, \dots, x^1 w_n + b_n\}$

• Regularisation: $\rightarrow$ reg term or require $\|W\| \leq \epsilon$

• **Dropout** method to avoid overfitting & do model averaging.

• explicitly running over several $\rightarrow$ **bagging**

• implicitly using whole network w/ reduced weights

• **Convolutional Nets**

$\rightarrow$ makes use of local LTI filters $\rightarrow$ sparse structure $\rightarrow$ fewer params

+ weight sharing (same filter at every position) $\rightarrow$ less params

+ padding (zero pad or valid pad)

+ multiple channels

• Convolution: $x^{(n)}[n] = \sum_k f[k] x^{(n-k)}$ [n-k mod] $\rightarrow$ cyclic $\rightarrow$ O(cL log(L)) w/ c small const

• Smoothness classifier: $g: x \rightarrow c$ $\rightarrow$ input size, filter, after padding

• **Adversarial ML** minimise $\mathcal{L}$

• Adversarial risk: $R(f, g) = \mathbb{E}_{x \sim p_D} [\max_{f \in \mathcal{F}} \mathbb{E}_{y \sim p_{Y|X}} [f(y, x)]]$

$\rightarrow$ how to find adversarial perturbation: if wrongly classified: $\tilde{x} = x + \epsilon h(x)$

• $\tilde{x}$ s.t. $\|x - \tilde{x}\| \leq \epsilon$ $\rightarrow$ $\tilde{x} = x - \epsilon h(x)$

• $\tilde{x}$ is the best perturbation: $\tilde{x} = x - \epsilon h(x)$

• **Randomised Smoothing:** making an existing classifier f robust

$\rightarrow$ smoothness classifier: $g: x \rightarrow c$ $\rightarrow$ max prob (c = f(x+z)) $\rightarrow$ $z \sim \mathcal{N}(0, \sigma^2)$

• **Unsupervised learning** learning without labels

use matrix factorisation to extract useful features, use clustering

1) **Representative learning**

2) **Density estimation & generative models**

• Define an unsupervised or self-supervised loss fct for:

• Compression & reconstruction e.g. Auto-Encoder

• Consistency & Contrastive Learning e.g. Noise-contrastive estimation

• Generation e.g. Auto-encoder, gaussian mixture model

• **Generative Adversarial Networks**

$\rightarrow$ optimisation of a GAN is formulated as a differentiable 2 player game where the generator G w/ params θ & the discriminator D w/ params ϕ aim at minimising their own cost fcts $\mathcal{L}_G, \mathcal{L}_D$:

• $\theta^* \in \arg\min_{\theta} \mathcal{L}_G(\theta, \phi^*)$ • $\phi^* \in \arg\max_{\phi} \mathcal{L}_D(\theta^*, \phi)$

$\rightarrow$ discriminative model: models posterior distribution $p(y|x)$

$\rightarrow$ generative model: models distribution $p(x)$ defined over x

$\Rightarrow$ Aim: converge to Nash equilibrium (NE) point (θ^*, ϕ^*)

• Nash Eq: $\mathcal{L}_G(\theta^*, \phi) \leq \mathcal{L}_G(\theta^*, \phi^*) \leq \mathcal{L}_D(\theta^*, \phi^*) \leq \mathcal{L}_D(\theta^*, \phi)$ $\forall \theta, \phi$

• **Differential NE** $\rightarrow$ $\| \nabla_{\theta} \mathcal{L}_G(\theta^*, \phi^*) \| = \| \nabla_{\phi} \mathcal{L}_D(\theta^*, \phi^*) \| = 0$

1) Discriminator "distinguishes" between real vs fake samples

2) Generator aims at fooling the discriminator that its samples are real

• **KL divergence:** $D_{KL}(P||Q) = \mathbb{E}_{x \sim P} [\log \frac{P(x)}{Q(x)}]$

• **JS divergence:** $D_{JS}(P||Q) = \frac{1}{2} D_{KL}(P||\bar{P}) + \frac{1}{2} D_{KL}(Q||\bar{P})$

• **Optimal discriminator:** $D^*(x) = \frac{P(x)}{P(x)+Q(x)}$

* K-Means clustering

Goal: find "prototype" points $\mu_1, \mu_2, \dots, \mu_k$ and cluster assignments $z_n \in \{1, 2, \dots, k\}$ $\forall x_n \in \mathbb{R}^D$

$\rightarrow$ Initialize μ_n $\forall k$, then iterate:

1) For all n , compute z_n given μ : $z_n = \arg\min_k \|x_n - \mu_k\|^2$

2) For all k , compute μ_k given z : $\mu_k = \frac{1}{n_k} \sum_{n: z_n=k} x_n$

$\rightarrow$ convergence to a local optimum is insured bc each step decreases the cost

• **Coordinate descent:** k-means is a coord. descent algo

$z^{(t+1)} := \arg\min_z \mathcal{L}(z, \mu^{(t)})$; $\mu^{(t+1)} := \arg\min_{\mu} \mathcal{L}(z^{(t+1)}, \mu)$

• **Matrix Factorization:** $\min_{Z, M} \mathcal{L}(Z, M) = \|X^T - MZ^T\|_{Fro}^2$

• **Issues with K-means:** $\rightarrow$ clusters are forced to be spherical

$\rightarrow$ computation can be heavy for large N, D and k

$\rightarrow$ each example can belong to only one cluster

• **Gaussian Mixture Models**

$\rightarrow$ Motivation: solve issues from k-means: ① elliptical clusters ② + 1pt $\rightarrow$ several clusters

① Clustering w/ gaussians: $p(x|\mu, \Sigma, \pi) = \prod_{n=1}^N \prod_{k=1}^K [\pi_k p(x_n|\mu_k, \Sigma_k)]^{\mathbb{1}_{z_n=k}}$

② Soft-Clustering: multinomial disks $p(z_n=k) = \pi_k$ $\sum_k \pi_k = 1$

$\rightarrow$ GMM: $p(x, z|\mu, \Sigma, \pi) = \prod_{n=1}^N p(z_n|\mu, \Sigma, \pi) p(x_n|z_n, \mu, \Sigma)$

• **Marginal likelihood:** $p(x_n|\theta) = \sum_{k=1}^K \pi_k p(x_n|\mu_k, \Sigma_k)$

$\rightarrow$ obtained by marginalising latent variable (z_n) out from likelihood

$\rightarrow$ good for statistical efficiency

$\Rightarrow$ w/o latent variable model, # of params grow at rate $O(N)$, after marginalization, growth reduced to $O(D^2 K)$

• **Maximum (marginal) likelihood:** $\max_{\theta} \mathcal{L}(\theta) = \sum_{n=1}^N \log \sum_{k=1}^K \pi_k p(x_n|\mu_k, \Sigma_k)$

$\rightarrow$ non-convex $\rightarrow$ not-unique $\rightarrow$ unbounded

• **Expectation-Maximisation (EM) Algorithm**

Start with $\theta^{(0)}$ and iterate:

1. Expectation step: compute a lower bound to the cost such that it's tight at the previous $\theta^{(t)}$

$\mathcal{L}(\theta) \geq \mathcal{L}(\theta, \theta^{(t)})$ and $\mathcal{L}(\theta^{(t+1)}) = \mathcal{L}(\theta^{(t)}, \theta^{(t)})$

2. Maximisation step: Update θ : $\theta^{(t+1)} = \arg\max_{\theta} \mathcal{L}(\theta, \theta^{(t)})$

• **Concavity of log:** $\log(\sum_{n=1}^N q_n r_n) \geq \sum_{n=1}^N q_n \log(r_n)$

• **EM for GMM:**

1. E-step: Compute assignments: $q_n^{(t)} = \frac{\pi_k p(x_n|\mu_k, \Sigma_k)}{\sum_{k=1}^K \pi_k p(x_n|\mu_k, \Sigma_k)}$

2. Compute the cost: $\mathcal{L}(\theta^{(t)}) = \sum_{n=1}^N \log \sum_{k=1}^K \pi_k p(x_n|\mu_k, \Sigma_k)$

3. M-step: Update: $\pi_k^{(t+1)} = \frac{1}{N} \sum_{n=1}^N q_n^{(t)}$

$\mu_k^{(t+1)} = \frac{\sum_{n=1}^N q_n^{(t)} x_n}{\sum_{n=1}^N q_n^{(t)}}$; $\Sigma_k^{(t+1)} = \frac{\sum_{n=1}^N q_n^{(t)} (x_n - \mu_k^{(t+1)})(x_n - \mu_k^{(t+1)})^T}{\sum_{n=1}^N q_n^{(t)}}$

• If $\Sigma_k = \sigma^2 I \rightarrow$ EM same as k-means as $\sigma^2 \rightarrow 0$

• $q_n^{(t)}$ is the posterior distribution of latent var: $q_n^{(t)} = p(z_n=k|x_n, \theta^{(t)})$

• EM in general: $\theta^{(t+1)} = \arg\max_{\theta} \sum_{n=1}^N \mathbb{E}_{\theta^{(t)}} [\log p(x_n, \theta)]$

• **SVD and PCA** $\rightarrow$ PCA is SVD for square matrices

• **PCA (Principal component analysis):** Dimensionality reduction: $f: D \rightarrow K$ $K \leq D$

$\rightarrow$ aim: compress data w/ minimal distortion

$\rightarrow$ mapping w/ max variance $\rightarrow$ "decompresses" input data

$\rightarrow$ singular values are eigenvalues of $X^T X$ w/ largest associated eigenvalue

$\rightarrow$ Singular value decomposition (SVD): $X = U \Sigma V^T$

$\rightarrow$ orthonormal $\Leftrightarrow U^T U = V^T V = I$

$\rightarrow$ $\|U\|_2 = 1$ $\rightarrow$ $\|V\|_2 = 1$ $\rightarrow$ $\|X\|_2 = \|\Sigma\|_2$

• Need 2 linear transforms for SVD:

1) C (the compression): $K \times D$ matrix

2) R (the reconstruction): $D \times K$ matrix $\Rightarrow \min \|X - RCX\|_F^2$

Lemma: $\|X - \hat{X}\|_F^2 \geq \|X - U\|_F^2$

$\rightarrow$ $X = U \Sigma V^T$, SVD of $X \rightarrow \Sigma$: singular values (diag entries) of X

$\rightarrow$ U : first k columns of U

• components are ordered in terms of importance

• best rank- k approx of X obtained by computing SVD & setting all singular vals to zero

• Note: $U^T U = I$ ($n \times n$); $V V^T = I$ ($D \times D$)

• **SVD and matrix factorisation** $X = U \Sigma V^T = W Z^T$

$\rightarrow$ assume D dim data points generated i.i.d according to unknown distrib D_x

$\rightarrow$ $\bar{x} = \frac{1}{N} \sum_{n=1}^N x_n$; $K = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})(x_n - \bar{x})^T$

• We can compute singular values of X by computing eigenvalues of $X X^T \rightarrow O(N^2)$

• If X symmetric $\rightarrow$ SVD: $U = V$, U, V non-necessarily positive entries

• If X positive semi-definite $\rightarrow$ all entries of Σ non-negative

• **Coordinate descent** Goal: Find $\alpha^* \in \mathbb{R}^N$ minimising $g(\alpha)$ or maximising $g(\alpha)$

• Idea: Update one coordinate at a time, while keeping others fixed

• **Kernel Ridge Regression & kernel trick**

• Ridge regression: $\min_w \frac{1}{2} \|y - Xw\|^2 + \frac{\lambda}{2} \|w\|^2$ Linear Kernel $K = X X^T$

$\rightarrow$ solution: $w^* = (X^T X + \lambda I_D)^{-1} X^T y$ $O(D^3 + ND^2)$

$\rightarrow$ kernelised: $w^* = X^T (X X^T + \lambda I_N)^{-1} y$ $O(N^3 + ND^2)$

• **Representer theorem:** for $\min_w \sum_{n=1}^N \mathcal{L}_n(x_n^T w, y_n) + \frac{\lambda}{2} \|w\|^2$

$\exists \alpha^*$ such that $w^* = X^T \alpha^*$ if we had α^* augmented the feature space to $\mathcal{B}(x)$

• **Augmented kernel:** $k = \sigma \sigma^T$ feature vector w/o extra costs

• **The kernel trick**

1) Define $K(x, x')$, $k_{11} = K(x_1, x_1)$ $\rightarrow k(x, x') = \sigma(x)^T \sigma(x')$

• $k(x, x') = x^T x'$ $\rightarrow \sigma(x) = x$ linear kernel

$\rightarrow \sigma(x') = x'$ $\rightarrow \sigma(x) = x^2$ polynomial kernel

$k(x, x') = (x_1 x'_1 + x_2 x'_2 + x_3 x'_3)^2$ $\rightarrow \sigma(x) = (x_1^2, x_2^2, x_3^2, \sqrt{2} x_1 x_2, \sqrt{2} x_1 x_3, \sqrt{2} x_2 x_3)$

$\rightarrow \sigma(x') = (x_1'^2, x_2'^2, x_3'^2, \sqrt{2} x_1' x_2', \sqrt{2} x_1' x_3', \sqrt{2} x_2' x_3')$

• $k(x, x') = a k_1(x, x') + b k_2(x, x')$ $\rightarrow \sigma(x) = (\sqrt{a} \sigma_1(x), \sqrt{b} \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x') + k_2(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$

• $k(x, x') = k_1(x, x')$ $\rightarrow \sigma(x) = (\sigma_1(x), \sigma_2(x))$